

JS-YAML - YAML 1.2 parser / writer for JavaScript buildunknown		Online Demo This is an implementation of YAML, a human-friendly data serialization language. Started as PyYAML port, it was completely rewritten from scratch. Now it's very fast, and supports 1.2 spec. Installation YAML module for node.js npm install js-yaml	CLI executable If you want to inspect your YAML files from CLI, install js-yaml globally: npm install -g js-yaml Usage usage: js-yaml [-h] [-v] [-c] [-t] file
1 8 well tested. Don't expect fast fixes if something goes wrong there. 2. <code>!!js/function</code> in browser bundle will not work by default. If you really need it - load <code>esprima</code> parser first (via <code>amd</code> or directly). 3. <code>!!bin</code> in browser will return <code>Array</code>, because browsers do not support <code>node.js</code> <code>Buffer</code> and adding <code>Buffer</code> shims is completely useless on practice. Notes: 1. We have no resources to support browserified version. Don't expect it to be	7 var doc = jsyaml.load('greeting: hello\nname: world'); </script> Browser support was done mostly for the online demo. If you find any errors - feel free to send pull requests with fixes. Also note, that IE and other old browsers needs es5-shims to operate. Notes: 1. We have no resources to support browserified version. Don't expect it to be	9 Bundled YAML library for browsers <!-- esprima required only for !!js/function --> <script src="esprima.js"></script> <script src="js-yaml.min.js"></script> <script type="text/javascript"> -t, --trace Show stack trace on error	5 Optional arguments: file File with YAML document(s) Optional arguments: -h, --help Show this help message and exit. -v, --version Show program's version number and exit. -c, --compact Display errors in compact mode
API Here we cover the most 'useful' methods. If you need advanced details (creating your own tags), see wiki and examples for more info. const yaml = require('js-yaml'); const fs = require('fs'); // Get document, or throw exception on error try { 9	const doc = yaml.safeLoad(fs.readFileSync('' home/ixti/example.yml', 'utf8')); console.log(doc); } catch (e) { console.log(e); } 10	safeLoad (string [, options]) Recommended loading way. Parses string as single YAML document. Returns either a plain object, a string or undefined, or throws <code>YAMLError</code> on error. By default, does not support <code>regexps</code>, functions and undefined. This method is safe for untrusted data. options: • filename (default: null) - string to be used as a file path in error/warning messages. 11	12 • onWarning (default: null) - function to call on warning messages. Loader will call this function with an instance of <code>YAMLError</code> for each warning. • schema (default: <code>DEFAULT_SAFE_SCHEMA</code>) - specifies a schema to use. ◦ <code>FAILSAFE_SCHEMA</code> - only strings, arrays and plain objects: http://www.yaml.org/spec/1.2/spec.html#id2802346 ◦ <code>JSON_SCHEMA</code> - all JSON-supported types: http://www.yaml.org/spec/1.2/spec.html#id2803231
16 const untrusted_code = "toString": i <tag:yaml.org,2002:js/function>"function() { (very_evil_thing());}" // I'm just converting that string, what could possibly go wrong? require('js-yaml').load(untrusted_code) +	15 load (string [, options]) Use with care with untrusted sources. The same as <code>safeLoad()</code> but uses <code>DEFAULT_FULL_SCHEMA</code> by default - adds some <code>JavaScript</code>-specific types: <code>!!js/function</code>, <code>!!js/regexp</code> and <code>!!js/undefined</code>. For untrusted sources, you must additionally validate object structure to avoid injections:	14 keys in a mapping will override values rather than throwing an error. NOTE: This function does not understand multi-document sources, it throws exception on those. NOTE: <code>JS-YAML</code> does not support schema-specific tag resolution restrictions. So, the <code>JSON</code> schema is not as strictly defined in the <code>YAML</code> specification. It allows numbers in any notation, use <code>Null</code> and <code>NULL</code> as <code>null</code>, etc. The core schema also has no such restrictions. It allows binary notation for integers.	13 • <code>CORE_SCHEMA</code> - same as <code>JSON_SCHEMA</code>: http://www.yaml.org/spec/1.2/spec.html#id2804923 ◦ <code>DEFAULT_SAFE_SCHEMA</code> - all supported <code>YAML</code> types, without unsafe ones (<code>!!js/undefined</code>, <code>!!js/regexp</code> and <code>!!js/function</code>): http://yaml.org/type/ ◦ <code>DEFAULT_FULL_SCHEMA</code> - all supported <code>YAML</code> types. • <code>JSON.parse</code> behaviour. If true, then duplicate <code>!!json</code> (default: false) - compatibility with <code>JSON</code>.

safeLoadAll (string [, iterator] [, options]) Same as safeLoad(), but understands multi-document sources. Applies iterator to each document if specified, or returns array of documents. <pre>const yaml = require('js-yaml'); yaml.safeLoadAll(data, function (doc) {</pre> 17	console.log(doc); }); loadAll (string [, iterator] [, options]) Same as safeLoadAll() but uses DEFAULT_FULL_SCHEMA by default. 18	safeDump (object [, options]) Serializes object as a YAML document. Uses DEFAULT_SAFE_SCHEMA, so it will throw an exception if you try to dump regexps or functions. However, you can disable exceptions by setting the skipInvalid option to true. options: indent (default: 2) - indentation width to use (in spaces). 19	- noArrayIndent (default: false) - when true, will not add an indentation level to array elements - skipInvalid (default: false) - do not throw on invalid types (like function in the safe schema) and skip pairs and single values with such types. - flowLevel (default: -1) - specifies level of nesting, when to switch from block to flow style for collections. -1 means block style everywhere 20
24 <pre> "binary" -> "0b1", "0b101010", "octal" -> "01", "052", "decimal" -> "1", "42", "7290", "hexadecimal" -> "0x1", "0x2A", "lowercase" => "true", "false" "uppercase" -> "TRUE", "FALSE" "camelcase" -> "True", "False" </pre>	23 <pre> "canonical" -> "~" "lowercase" => "null" "uppercase" -> "NULL" "camelcase" -> "Null" </pre> The following table show available styles (e.g. "canonical", "binary"...), available for each tag (e.g. !null, !int...). Yaml output is shown on the right side after => (default setting) or ->: <pre> !int </pre>	22 - noCompatMode (default: false) - if true don't try to be compatible with older yaml versions. Currently: don't quote "yes", "no" and so on, as required for YAML 1.1 - condenseFlow (default: false) - if true flow sequences will be condensed, omitting the space between a, b. Eg. '[a,b]', and omitting the space between key: value and quoting the key. Eg. '{a":b}' Can be useful when using yaml for pretty URL query params as spaces are %-encoded.	21 - styles - "tag" => "style" map. Each tag may have own set of styles. - schema (default: DEFAULT_SAFE_SCHEMA) - sortKeys (default: false) - if true, sort specifies a schema to use. - keys when dumping YAML. If a function, use the function to sort the keys. - lineWidth (default: 80) - set max line width. - noRefs (default: false) - if true, don't convert duplicate objects into references
25 <pre> !!float "lowercase" => ".nan", '.inf' "uppercase" -> ".NAN", '.INF' "camelcase" -> ".NaN", '.Inf' </pre> Example: <pre>safeDump (object, { 'styles': { '!!null': 'canonical' // dump null as ~ }, </pre>	26 <pre> // sort 'sortKeys': true object keys </pre> dump (object [, options]) Same as safeDump() but without limits (uses DEFAULT_FULL_SCHEMA by default).	27 Supported YAML types The list of standard YAML tags and corresponding JavaScript types. See also YAML tag discussion and YAML types repository. <pre> !!null '' # null !!bool 'yes' # bool !!int '3...' # number !!float '3.14...' # number !!binary '...base64...' # buffer !!timestamp 'YYYY-...' # date !!omap [...] # array of </pre>	28 key-value pairs <pre>!!pairs [...] # array or array pairs !!set { ... } # array of objects with given keys and null values !!str '...' # string !!seq [...] # array !!map { ... } # object</pre> JavaScript-specific tags
32 The maintainers of js-yaml and thousands of other packages are working with Tidelft to deliver commercial support and maintenance for the open source dependencies you use to build your applications. Save time, reduce risk, and improve code health, while paying the maintainers of the exact dependencies you use. Learn more.	31 Also, reading of properties on implicit block mapping keys is not supported yet. So, the following YAML document cannot be loaded. <pre> &anchor foo: foo: bar anchor: duplicate key baz: bat anchor: duplicate key </pre> Available as part of the Tidelft Subscription	30 <pre> { "foo,bar": ["baz"], "object object["baz", "baz"] } </pre> adding them. toString() method) them at the moment of	29 Caveats Note, that you use arrays or objects as key in JS-YAML. JS does not allow objects or arrays as keys, and stringifies (by calling <pre> i.js/regexp /pattern/gim # Regexp i.js/undefined '' # i.js/function 'function () {...}' # Function </pre>